Лекция 5

NumPy

Давыдов Виталий Валерьевич (ГАИШ МГУ)

Проект Jupyter (jupyter.org)

- Инструменты и **стандарты** для интерактивных вычислений с использованием интерактивных блокнотов (computational notebooks)
- Computational Notebook – документ, включающий компьютерный код, данные, визуализации (изображения и графики), интерактивные элементы управления, форматированный текст (формулы).
- Включает множество проектов
  - Jupyter User Interfaces (JupyterLab, Jupyter Notebook, Jupyter Desktop, Jupyter Console…)
  - Kernels (процессы, реализующие интерпретаторы языков программирования)
  - Ipython
  - https://jupyter4edu.github.io/jupyter-edu-book/
- https://nbviewer.org/

Computational Notebook

- Последовательность ячеек (cells)
- Есть различные типы ячеек:
  - Code (исполняемый код)
  - Markdown (форматированный текст)
  - Raw (неформатированный текст)
- Код исполняется в Kernel
  - Результат отображается в ячейке под кодом

[5]:
```
import matplotlib.pyplot as plt
plt.style.use('classic')
%matplotlib inline
import numpy as np
import pandas as pd
import seaborn as sns
sns.set()
```

[6]:
```
rng = np.random.RandomState(0)
x = np.linspace(0, 10, 500)
y = np.cumsum(rng.randn(500, 6), 0)
```

Next step

Now, create a graph.

[7]:
```
plt.plot(x, y)
plt.legend('ABCDEF', ncol=2, loc='upper left');
```



Возможности Jupyter

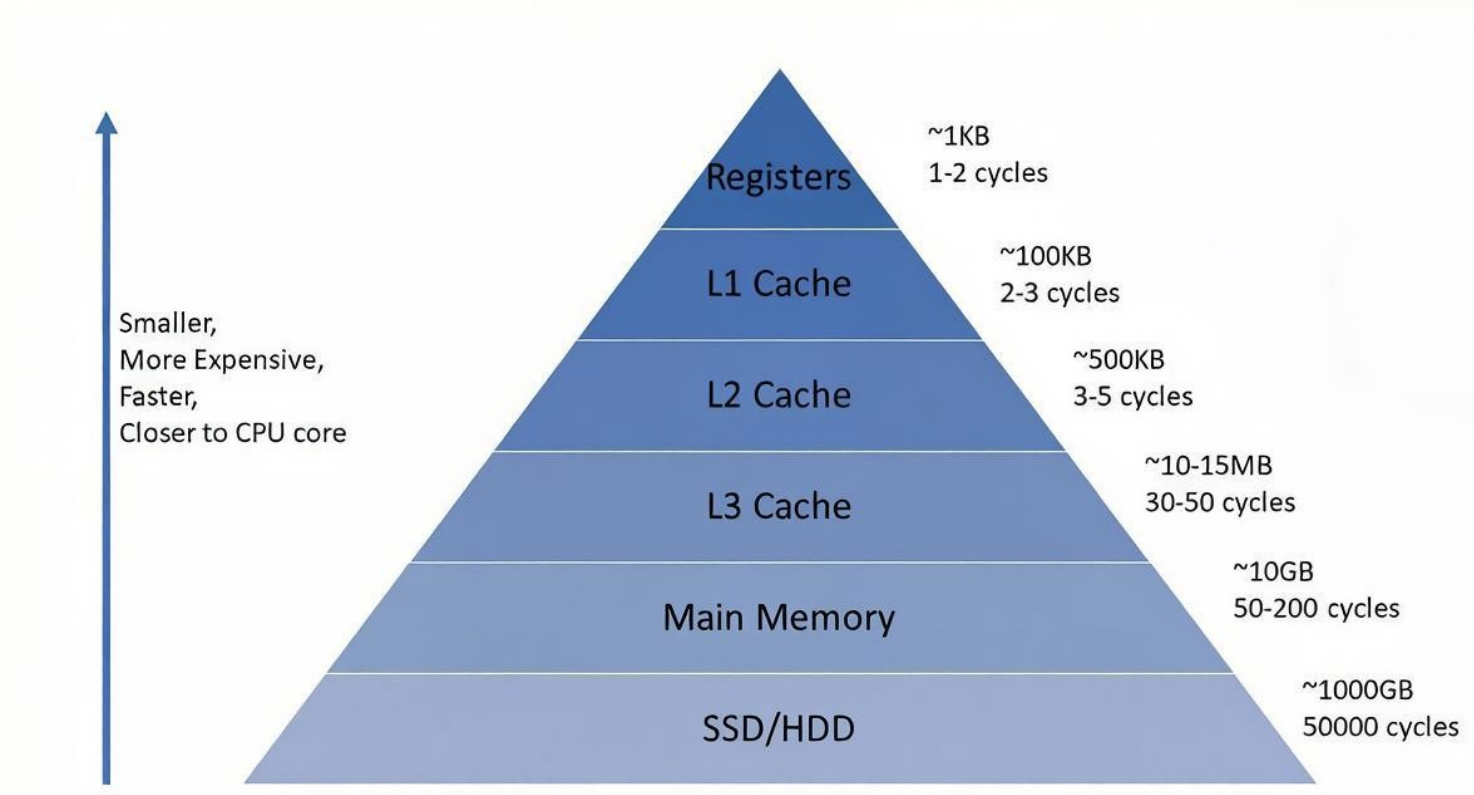
- Отображение кода и результата в одном окне с возможностью интерактивного редактирования кода
- Отображение визуализаций (графиков, изображений), интерактивных элементов управления
- Форматированный текст (markdown) и структурирование
- Отображение формул (latex)
- Экспорт в другие форматы (pdf, html)
- Документация:
  - https://jupyterlab.readthedocs.io
  - https://jupyter-notebook.readthedocs.io/

Полезные фичи

- Клавиатурные комбинации:
  - Выполнить одну ячейку : CTRL + ENTER
  - Добавить новую ячейку: ESC a (above), ESC b (below)
  - Удалить текущую ячейку: ESC d d
- Тайминг выполнения одной строки, ячейки: %timeit, %%timeit
- Выполнить скрипт: %%bash
- Исполнить ячейку как latex, html: %%latex, %%html
- Установить точность вывода FP (не в print): %precision
- Информация по "магическим" командам: %magic

Память

Структура уровней памяти компьютера



Тайминг

Level	Typical Size	Access Time	Managed By	Volatile?
Registers	~1 KB	< 1 ns	Compiler	Yes
L1 Cache	32–64 KB	1–4 ns	Hardware	Yes
L2 Cache	256 KB–1 MB	4–12 ns	Hardware	Yes
L3 Cache	4–64 MB	10–50 ns	Hardware	Yes
Main Memory (RAM)	4–128 GB	50–100 ns	OS	Yes
SSD	256 GB–4 TB	50–150 µs	OS	No
HDD	1–20 TB	5–10 ms	OS	No
Tape/Optical	TB–PB	seconds–minutes	OS/Manual	No

Схема КЭШ памяти процессоров (x86_64)

Кэш-память

Схема иерархии кэш-памяти процессора:

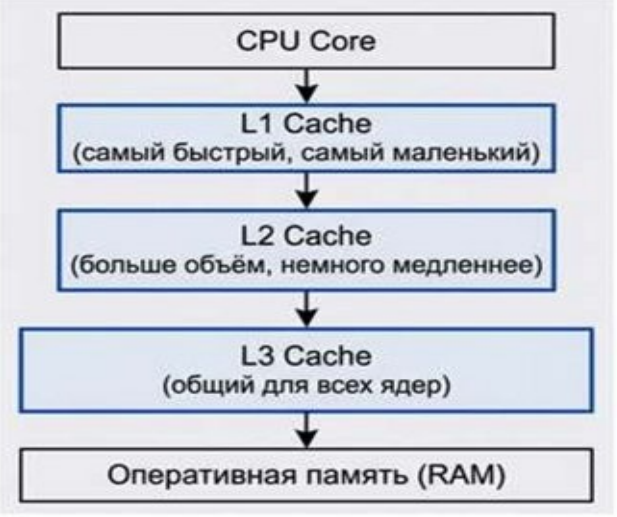
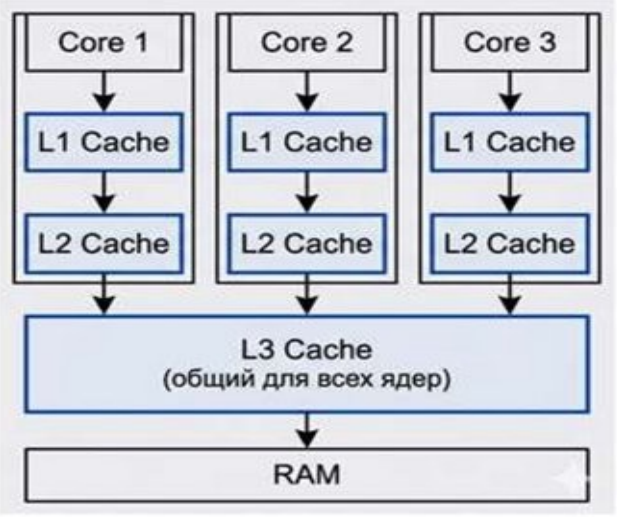


Схема кэша в многоядерном процессоре



Numpy

Пакет NumPy

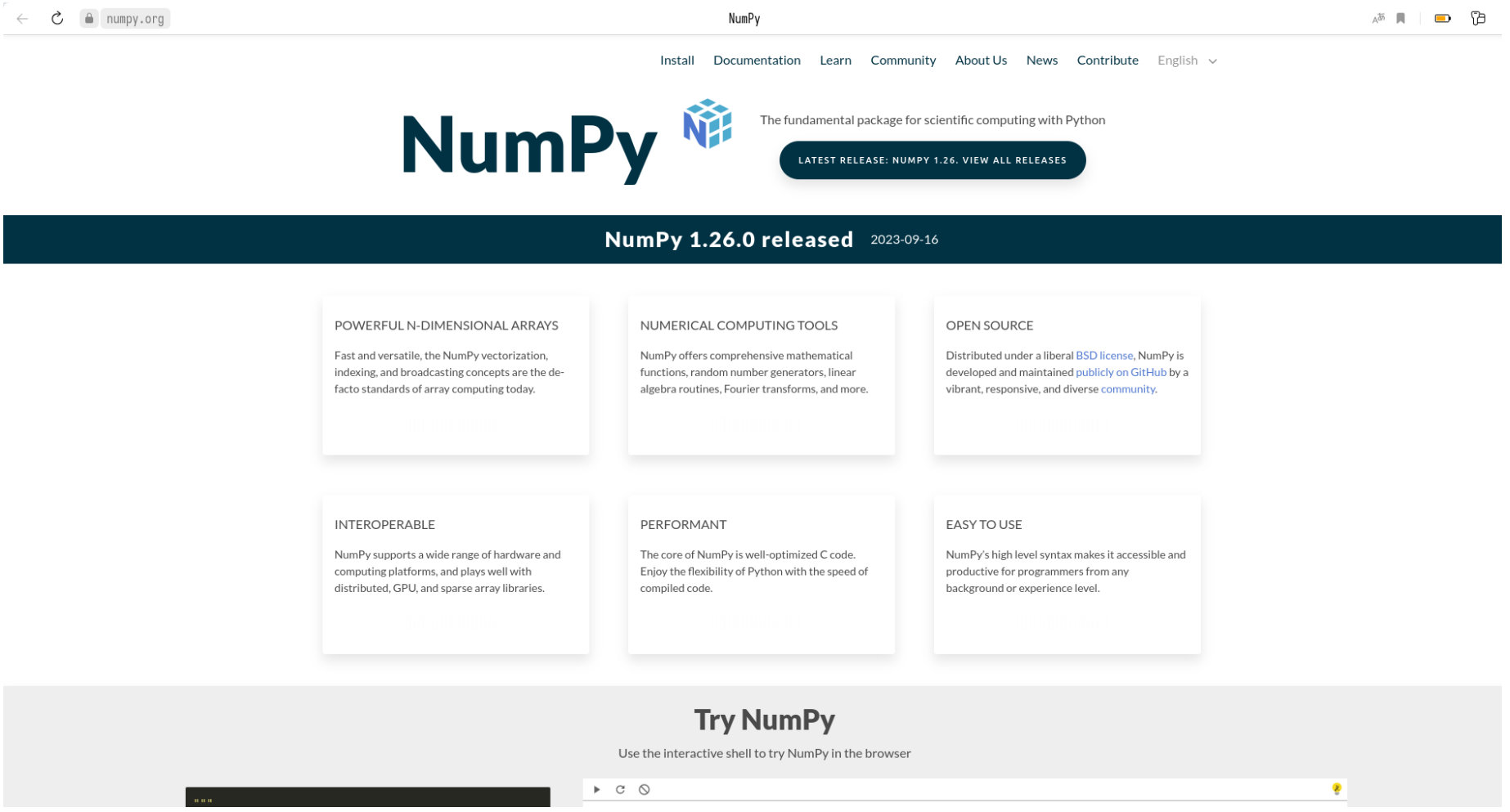
Библиотека для работы с многомерными массивами:

- Многомерный массив (numpy.ndarray)
- Новые скалярные типы, расширяющие систему типов Python (знаковые и беззнаковые)
- Masked array (numpy.ma)
- Типы для работы со временем (numpy.datetime64)

Набор операций над массивами:

- Создание и изменение многомерных массивов
- Математические функции
- Загрузка и сохранение данных на диск
- Операции линейной алгебры
- Решение линейных уравнений
- Операции с маскированными массивами
- Математические функции (применяемые поэлементно)
- Полиномы и статистики

Сайт numpy.org



Скалярные типы

Basic Type	Available NumPy types	Comments
Boolean	bool	Elements are 1 byte in size
Integer	int8, int16, int32, int64, int128, int	int defaults to the size of int in C for the platform
Unsigned Integer	uint8, uint16, uint32, uint64, uint128, uint	uint defaults to the size of unsigned int in C for the platform
Float	float32, float64, float, longfloat,	Float is always a double precision floating point value (64 bits). longfloat represents large precision floats. Its size is platform dependent.
Complex	complex64, complex128, complex	The real and complex elements of a complex64 are each represented by a single precision (32 bit) value for a total size of 64 bits.
Strings	str, unicode	Unicode is always UTF32 (UCS4)
Object	object	Represent items in array as Python objects.
Records	void	Used for arbitrary data structures in record arrays.

Многомерный массив (пример)

Создание многомерного массива из последовательностей Python:

np.array([1, 2, 3, 4, 5]) – одномерный массив

np.array([[11, 12], [21, 22]]) – двухмерный массив

np.array(range(100)) - одномерный массив из генератора

Операции над массивами:

- Операции выполняются поэлементно
- Массивы можно передавать в математические функции numpy

numpy.ndarray

Многомерный массив значений одного типа

- Размер задается во время создания
- Изменение размера приведет к удалению старого массива и созданию нового
- Поэлементное изменение данных массива
- Элементы массива одного типа (размера)
- Многие операции реализованы в скомпилированном коде
- Поддержка больших массивов
- Другие научные пакеты для Python используют массивы numpy
- Расширенный диалект языка Python для работы с массивами

Основные свойства

- dtype (data type, тип значений)
- shape (форма, количество элементов по каждому измерению)
- ndim (количество измерений)
- size (общее количество элементов)
- base (ссылка на исходные данные для представлений или None)

Полный список свойств и методов ndarray:

- https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html

Создание многомерных массивов

Задание формы (shape) массива (N-Dim)

np.**empty**([10, 10], dtype=np.float64) - неинициализированный массив
np.**zeroes**(...) - массив с нулями
np.**ones**(...) - массив из единиц

Из последовательностей Python (1-Dim)

np.array(<list>), np.array(<tuple>)

С использованием генераторов (1-Dim)

np.arange([start,] stop[, step,][, dtype, like])
np.linspace(start, stop[, num, endpoint, ...])

Из внешних источников

np.fromfile(...), np.fromfunction(...), np.fromiter(...), np.fromstring

Матрицы (2-Dim)

np.diag, np.tri, np.vander, np.mat(data, dtype)

Доступ к элементу по индексу

- Индекс элемента – тапл, содержащий позицию по каждой оси
  - Общая форма: X[<obj>], где obj – list, tuple, slice (срез), генератор и др
- Одномерные массивы
  - x[10] - элемент по индексу 10
  - x[(10)]
- Двухмерные массивы
  - x[0] - возвращает срез массива по оси 0 (view)
  - x[1,3] - возвращает значение элемента
  - x[[(0,0), (0,1), (10,10)]]
- Многомерные массивы
  - x[1,2,3,4,5] - в случае 5 осей доступ к элементу (1,2,3,4,5)
- Нет операции удаления элемента в существующем массиве
  - numpy.delete(arr, obj, axis=None) – создает новый массив

Срезы (продолжение)

- Фильтрация элементов
  - x = np.array([1., 2., np.nan, 3., np.nan, np.nan])
  - y = x[~np.isnan(x)]
- Операции по условию
  - x = np.array([1., -1., -2., 3])
  - x[x < 0] += 100

Представления (views)

- b = a[1:5], где b.base != None (b – представление)
- b = a[1:5].copy(), где b.base == None (копия)
- Представления разделяют тот же блок памяти
- Могут иметь shape, отличный от оригинала
- Используются для уменьшения расходования памяти

Broadcasting

- Набор правил для выполнения арифметических операций с массивами разных размерностей
- Массив с меньшей размерностью "расширяется" до размерности другого массива
- Правила броадкастинга:
  - Перед выполнением бинарной операции идет сравнение размерностей двух массивов
  - Размерности сравнимы, если: (1) они одинаковы, (2) одна из них равна 1
  - Если размерности не сравнимы, выдается ошибка ValueError: operands could not be broadcast together
  - К-во размерностей результата – к-во размерностей "большего" массива

```
» a = np.array([1.0, 2.0, 3.0])
» b = 2.
» a * b
array([2.,  4.,  6.])
```

```
Image  (3d array): 256 x 256 x 3
Scale  (1d array):             3
Result (3d array): 256 x 256 x 3
```

Универсальные Функции (ufunc)

- Оперирует массивами поэлементно
- Поддерживает броадкастинг, приведение типов
- Реализуют векторизацию

```
x = [1, 2, 3, 4]
y = [4, 5, 6, 7]
Z = []

for i, j in zip(x, y):
  z.append(i + j)

print(z)
```

```
import numpy as np

x = [1, 2, 3, 4]
Y = [4, 5, 6, 7]

z = np.add(x, y)

print(z)
```

Пример: Линейный МНК

- Практическая работа в Jupyter Lab